

Theory Questions

Question 1:

Define "Query Optimization." Why is the term "optimization" slightly misleading in the context of database management systems?

Question 2:

What is the primary motivation for query optimization from the perspective of an application developer versus the DBMS?

Question 3:

State the two general principles of query optimization regarding disk I/O and memory consumption.

Question 4:

What are the main categories of "Physical Optimizations" that a DBMS might employ after the initial query tree is determined?

Question 5:

If a query involves a join among several relations, the number of possible join orderings is significant. Why does the number of alternative evaluation strategies increase exponentially when you consider implementation choices (e.g., nested-loop vs. hash join) and pipelining?

Question 6:

Describe the process of "Heuristic-based" optimization versus "Cost-based" optimization.

Question 7:

Why does a DBMS maintain "Histograms" for attributes in a database? How do these histograms assist the Query Optimizer?

Question 8:

What are the three main cost factors (aside from total time) that a Query Optimizer attempts to minimize?

Question 9:

Explain the concept of "Selection Pushing" and why it is a classic rule-based optimization technique.

Question 10:

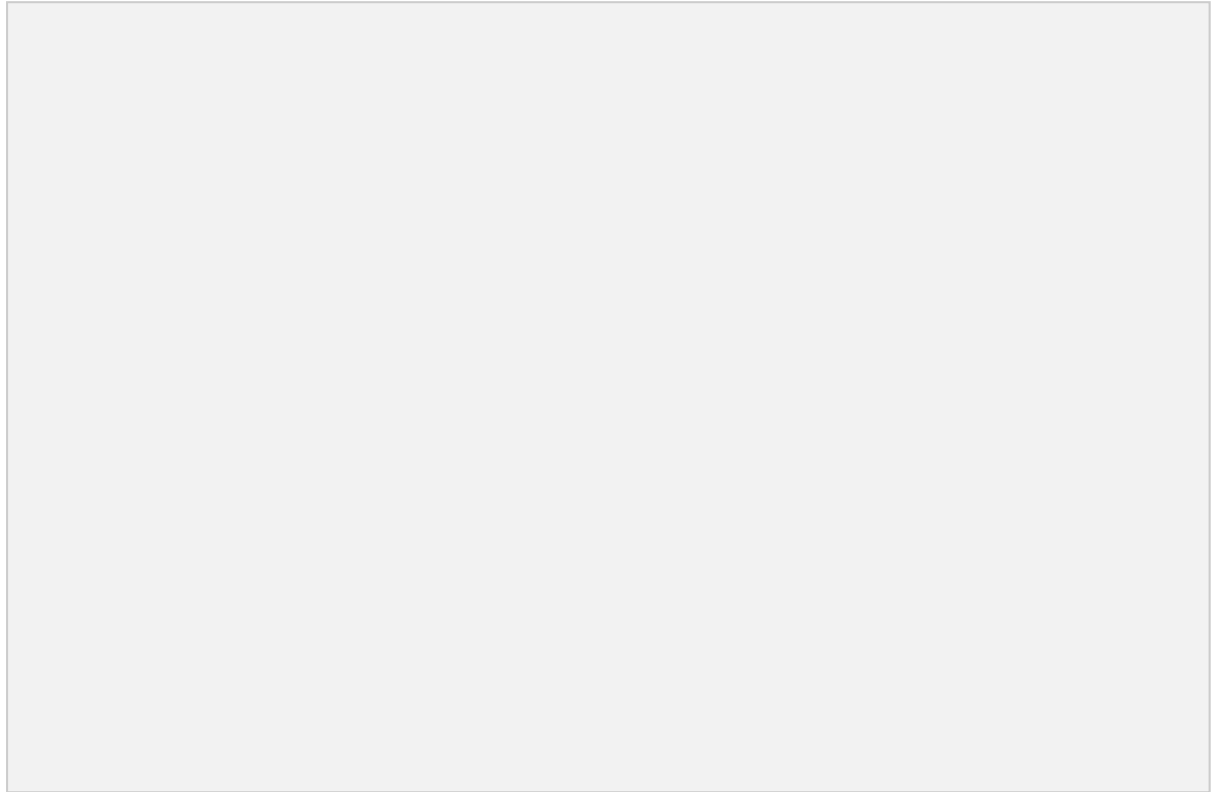
What is "Query Plan Caching," and why is it beneficial for systems that receive a high volume of similar queries?

Question 11:

If a DBMS has multiple ways to execute a query, why might it choose a plan that is not the *absolute* fastest one? (Hint: Consider the overhead of the optimization process itself).

Algebraic Optimization Questions:

Assume the following is the relational schema for all questions in this category.



Question 1:

Draw the query tree for the following SQL query then optimize the tree assuming breaking select operations and pushing them downwards in the tree is better to filter records early and, in addition to that, projecting as early as possible is better to hold smaller records in memory during query execution. Then write the relational algebra expression for your modified query tree.

```
Select Fname, Lname, Ssn
  from EMPLOYEE
  where address = 'Merul Badda' and Salary > 50000
```

Question 2:

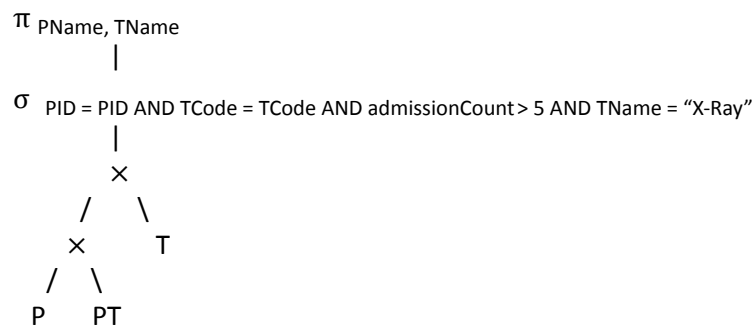
Draw the query tree for the following SQL query then optimize the tree assuming the conditions of Question 1 and write down the relational algebra expression for your final query.

```
Select Dname, Fname, LName
  From Department join Project join Employee
    On Department.Dnumber = Project.Dnum
    and Department.Mgr_Ssn = Employee.Ssn
  where Project.Location = 'BRAC University'
    and Employee.Address = 'Gulshan 1, Dhaka'
    and Department.Mgr_start_date < '01/01/2020'
```

Question 3:

Hospital Database Schema: Patient (<u>PID</u>, PName, address, ward, admissionCount) -- PK: PID PatientTreatment (<u>PID</u>, <u>TCode</u>, Result) -- PK: PID, TCode -- FK: PID references Patient(PID) -- FK: TCode references Treatment(TCode) Treatment (<u>TCode</u>, TName) -- PK: TCode	<pre>SELECT P.PName, T.TName FROM Patient P, PatientTreatment PT, Treatment T WHERE P.PID = PT.PID AND PT.TCode = T.TCode AND P.admissionCount > 5 AND T.TName = "X-Ray";</pre>
--	--

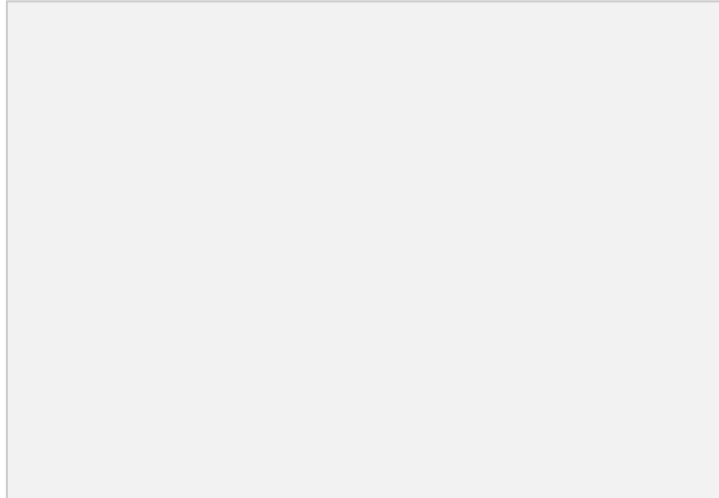
From the given query we get the following query tree.



Optimize the query tree with algebraic modifications.

Cost-based Optimization Questions:

Assume the following catalog Information is available for the database of the 'Algebraic Modification Questions' section. Further, assume indexing is applied on the primary keys of the relations only.

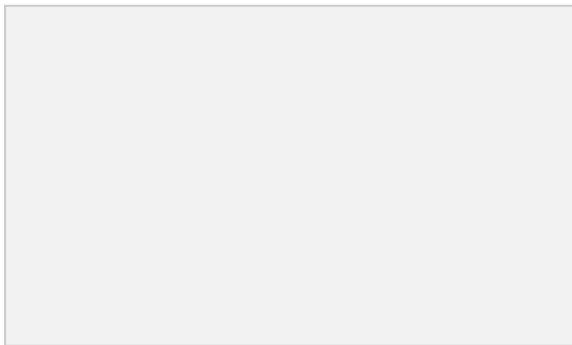


Question 1:

From the database catalogue information above, decide if a query join Project, Employee, and Departments; in general, which relation's records should be retrieved and joined earlier and why.

Question 2:

In addition to the catalogue if a histogram on Employee relation's salary attribute is as follows



Then in the following query about Employees, which selection is more critical and which relation's records should be retrieved first for checking the join condition for best performance?

```
Select Fname, Lname
  from EMPLOYEE join PROJECT
        on Essn = Ssn
 where salary > 80000 and Pnumber > 1000
```

Justify your answer.

Other Critical Reasoning Questions:

Question 1:

You are managing a database for a large e-commerce platform. You have two primary tables: Users (1 million records) and Orders (50 million records). Users frequently run a query to find their own order history:

```
SELECT * FROM Orders WHERE user_id = ? ORDER BY order_date DESC;
```

The database is currently performing a full table scan on the Orders table, which is causing significant latency.

- **Question:** As a database administrator, what two specific optimization strategies would you implement to improve the performance of this query? Justify your choices using the principles of Query Optimization (e.g., Disk I/O reduction, Indexing).

Question 2:

You have a complex reporting query that joins three tables: Sales, Products, and Regions.

- Sales: 10,000,000 rows
- Products: 5,000 rows
- Regions: 50 rows
- The query includes filters for Region = 'North America' and Product_Category = 'Electronics'.
- **Question:** Explain how a Cost-Based Optimizer would prioritize the join order. Which table would it likely join first, and why? Explain the concept of "Selection Pushing" in the context of this specific scenario to minimize intermediate results.

Solutions:

Theory Questions

Answer 1:

Query optimization is the process of converting a declarative SQL query into an efficient execution plan. It is "misleading" because the system does not guarantee the *mathematically optimal* path; it merely searches for a "good enough" or significantly improved execution plan within a reasonable time.

Answer 2:

Developers write queries declaratively (focusing on the *what*). The DBMS must figure out *how* to ensure the system remains responsive for all users, preventing a single inefficient query from consuming excessive system resources.

Answer 3:

1. Reduce disk I/O: Data retrieval from the disk is the most expensive operation.
2. Filter data early: Apply selection/projection as early as possible to minimize the volume of data kept in memory during operations like joins.

Answer 4:

Physical optimizations include: bottom-up or top-down execution strategies, sharing intermediate results between queries, parallel execution using multiple threads/processes, and caching query plans for reuse.

Answer 5:

Every unique join order creates a new branch of possibilities. When you multiply those orderings by different algorithmic implementations (e.g., hash join vs. merge join) and add the choice of pipelining versus materialization, the search space for the "best" plan grows geometrically, often reaching hundreds of combinations for even simple multi-table joins.

Answer 6:

Heuristic optimization uses static rules (like "always push selections down"). Cost-based optimization uses statistical data (histograms) to estimate the cost of various plans in terms of disk I/O, CPU, and memory, selecting the one with the lowest estimated cost.

Answer 7:

Histograms store the distribution of data values in a column. They help the optimizer estimate the "selectivity" of a predicate—essentially predicting how many rows will be returned by a condition—which is crucial for choosing the right join order.

Answer 8:

1. Storage Access (Disk I/O).
2. Buffer consumption (Memory usage).
3. Computation time (CPU cycles for sorting/hashing).

Answer 9:

"Selection Pushing" moves filter operations (WHERE clauses) down the query tree to be performed as close to the data source as possible. This reduces the number of tuples moving up the tree, making subsequent joins much faster.

Answer 10:

Query Plan Caching stores the optimized execution plan of a previously executed query. If a similar query arrives later, the DBMS can skip the expensive optimization phase and reuse the existing plan.

Answer 11:

Finding the *perfect* optimal plan is an NP-hard problem. The time spent optimizing a query is also a "cost." If a query is very simple, the system chooses a "good" plan quickly rather than spending excessive CPU time searching for the "best" one.

Algebraic Optimization Problems

Answer 1:

Skipping the tree drawing part. Here is the final query in relational algebra format.

$$E1 = \pi_{Fname, Lname, Ssn, Address, Salary} (Employee)$$

$$E2 = \sigma_{salary > 50000} (\sigma_{address = 'Merul Badda'} E1)$$

$$E3 = \pi_{Fname, Lname, Ssn} (E2)$$

Answer 2:

Skipping the tree drawing part. Here is the final query in relational algebra format.

$$E1 = \sigma_{Address = 'Gulshan 1, Dhaka'} (\pi_{Fname, Lname, Ssn, Address} (Employee))$$

$$E2 = \sigma_{Location = 'BRAC University'} (\pi_{Location, Dnum} (Project))$$

$$E3 = \sigma_{Mgr_start_date < '01/01/2020'} (\pi_{Dnumber, Dname, Mgr_Ssn, Mgr_start_date} (Department))$$

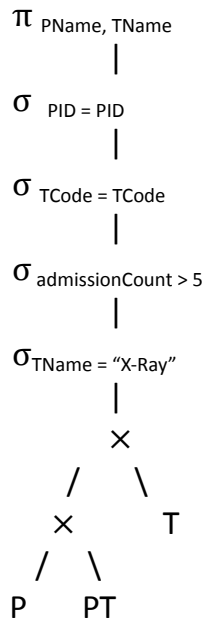
$$E4 = E3 \bowtie_{Mgr_Ssn = Ssn} E1$$

$$E5 = E2 \bowtie_{Dnum = Dnumber} E4$$

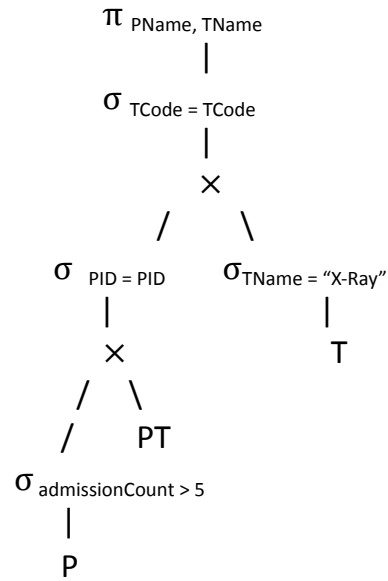
$$E6 = \pi_{Fname, Lname, Dname} (E5)$$

Answer 3:

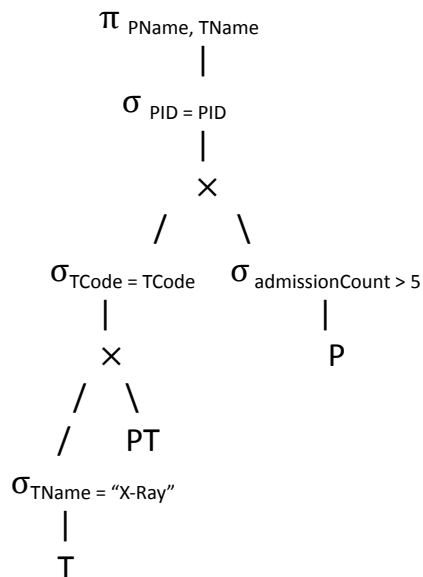
Step 1: Cascade of selection



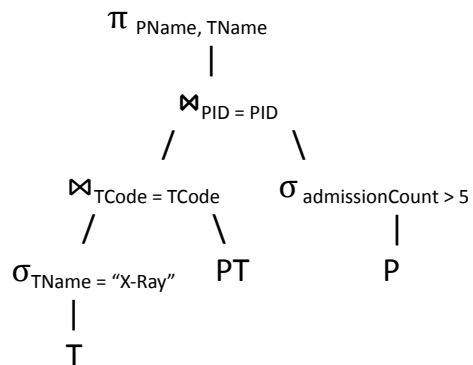
Step 2: pushdown selection to leaf



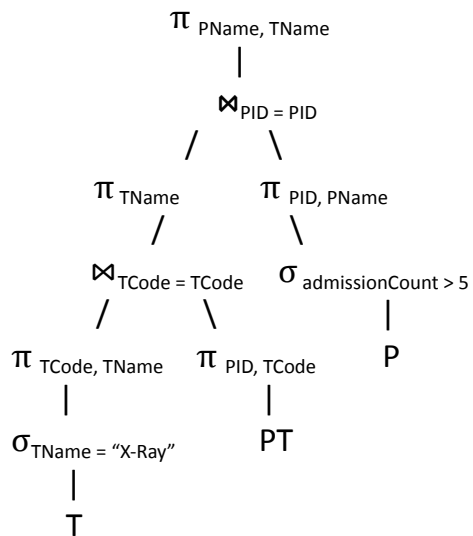
Step 3: apply restrictive select first



Step 4: Convert X into ⋈



Step 5: inject projection



Cost-based Optimization Questions:

Answer 1:

From the catalogue information alone, we see that the number of records for the Department relation is significantly smaller than that for the Employee and Project relations. Hence, it is better to retrieve and join records of the Department relation first.

Answer 2:

According to the histogram on the Employee relation, there are only a few employees with salary higher than 80k. On the other hand, if we consider uniform distributions for project numbers, we will have around 1k records from the Project relation. Therefore, in the given query, it is better to push the selection on the employee on salary attribute downwards in the query tree and retrieve Employee records earlier for doing the join.

Other Critical Reasoning Questions

Answer 1:

1. **Create a Composite Index:** You should create an index on (user_id, order_date). This allows the DBMS to perform an index-based search to immediately isolate the rows for a specific user (reducing disk I/O from 50 million records to just the relevant ones) and utilizes the order_date part of the index to satisfy the ORDER BY clause without requiring an additional, expensive sorting operation.
2. **Selection Pushing/Filtering:** Ensure the query engine uses the index to filter by user_id *before* fetching the full row data. This adheres to the principle of "applying filtering as early as possible," ensuring that only the necessary records are pulled into memory.

Answer 2:

- **Join Order Strategy:** A Cost-Based Optimizer would likely join the smallest tables first. It would start by joining Regions (50 rows) with the filtered subset of Sales (using the 'North America' criteria), then join the result with Products. Joining smaller tables first drastically reduces the size of the intermediate result set that must be processed in memory.
- **Selection Pushing:** In this scenario, "Selection Pushing" means applying the filters (Region = 'North America' and Product_Category = 'Electronics') *before* the join operations occur. By applying these filters at the leaf level of the query tree, the DBMS significantly reduces the volume of Sales data that needs to be joined with Products and Regions, which effectively minimizes buffer consumption and computation time.